

Especificación de Diseño

Gestor de Sueldos "25 de Mayo"

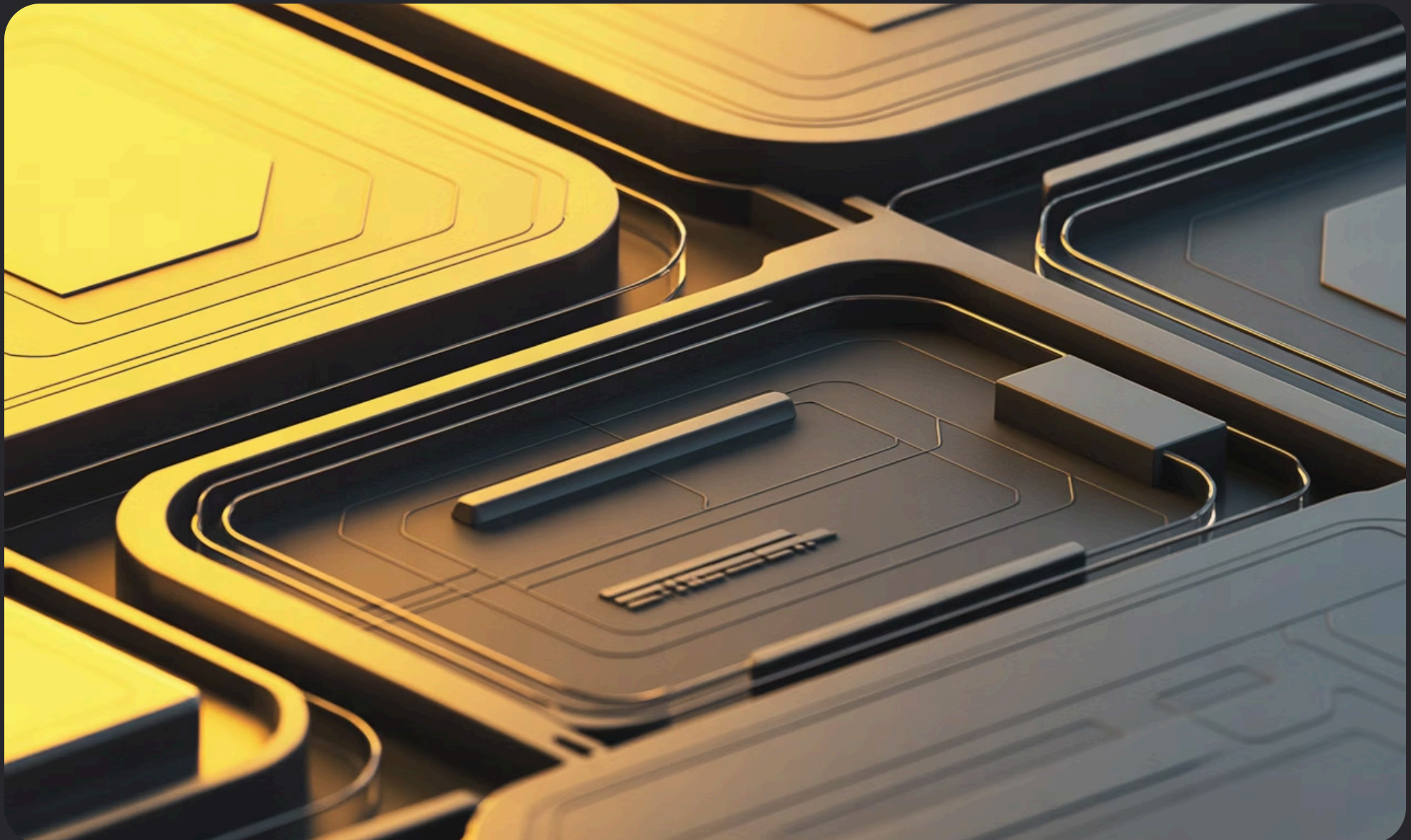


**COOPERATIVA DE
SERVICIOS PÚBLICOS**
25 de Mayo
Ltda.

Propósito del Documento

La Especificación de Diseño describe cómo está construido el sistema, qué arquitectura utiliza, cómo se organizan sus componentes, cómo se comunican entre sí, y cómo se garantiza la consistencia del proceso de liquidación de sueldos

Este documento detalla cómo lo hace a nivel técnico y estructural.



1. Arquitectura General del Sistema

El Gestor utiliza una arquitectura cliente-servidor dividida en dos aplicaciones:

Frontend

Desarrollado en ReactJS, responsable de la interfaz gráfica, la interacción del usuario, los formularios, visualización de datos, páginas, componentes y comunicación HTTP con el backend mediante Axios.

Backend

Desarrollado en Java 17 + Spring Boot 3.5, responsable de la API REST, la lógica de negocio, el cálculo salarial, las validaciones, la persistencia en PostgreSQL, y la gestión del modelo de datos.

Base de Datos

Motor: PostgreSQL

Administración de tablas, relaciones y restricciones con Hibernate ORM.

3 - Diseño del Backend (Capa Servidor)

3.1 Patrón de Arquitectura (Controller → Service → Repository → Entity)

El backend utiliza el patrón típico de Spring Boot dividido en capas:

01

Controller

Expone los endpoints REST. Recibe solicitudes desde el frontend, valida datos básicos y pasa el trabajo al servicio correspondiente.

Ejemplos: EmpleadoController, ConvenioController



Capa de Servicio

2 - Service

Contiene la lógica de negocio real. Aquí se realizan los cálculos complejos (como la liquidación del sueldo).

- Validaciones internas.
- Interacciones con repositorios.
- Ensamblado de DTOs.

Ejemplos: LiquidacionSueldosService, EmpleadoService, ConvenioService



Capa de Repositorio

3 - Repository

Interfaz que extiende JpaRepository para acceso a base de datos. No contienen lógica, solo acceso CRUD

Ejemplos:

- EmpleadoRepository
- ConvenioRepository
- PagoSueldoRepository
- PagoConceptoRepository



Modelo de Dominio

4 - Entities (Modelo de Dominio)

Son las clases que representan tablas en la base:



Empleado



Convenio



Categoria



Zona



PagoSueldo



PagoConcepto



Descuento



BonificacionFija

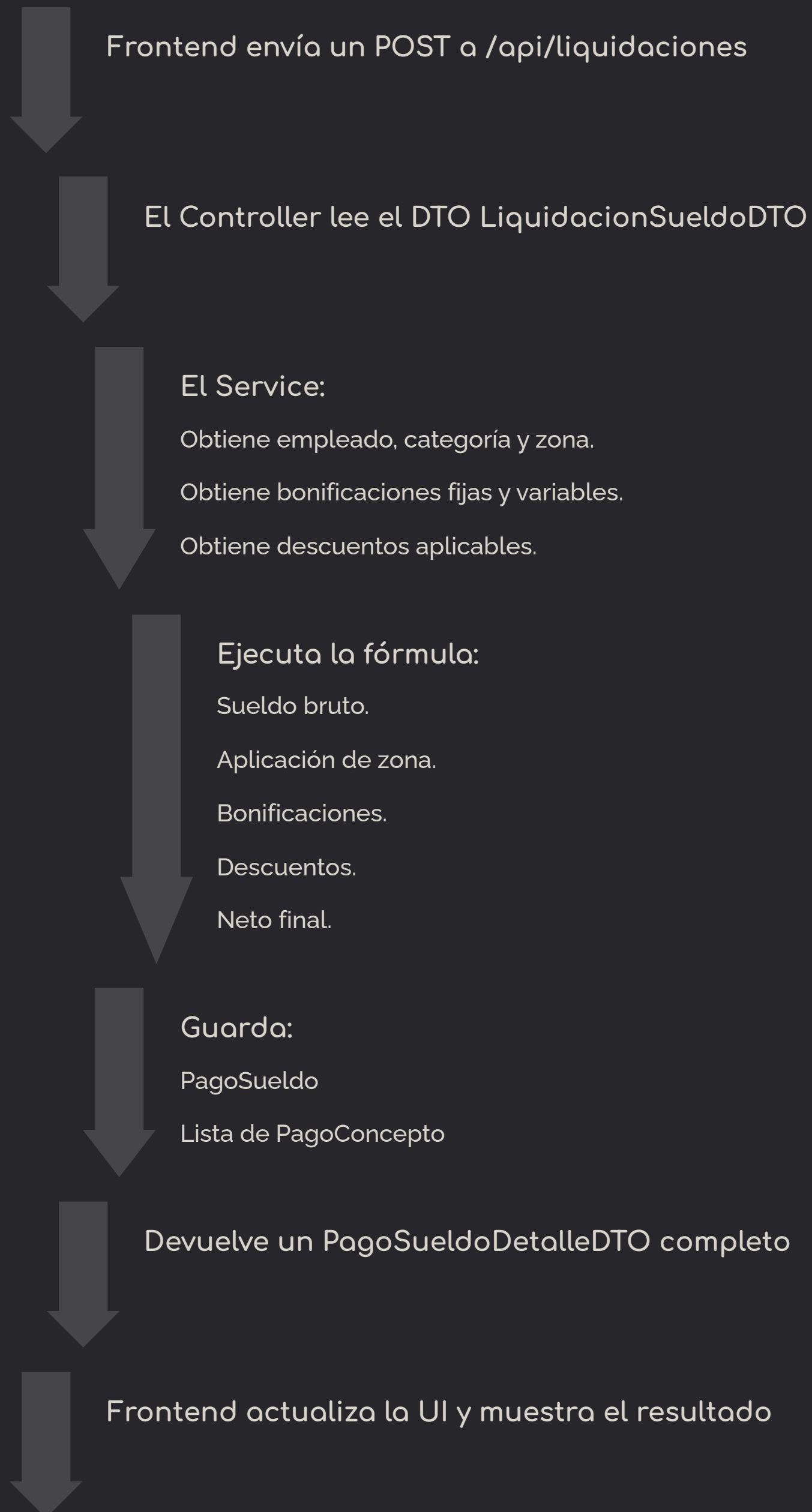


BonificacionVariable

El ORM de Hibernate gestiona la creación, relaciones (Una a Muchas y Muchas a Una) y claves foráneas.

3.2 Flujo de un endpoint

Cuando el usuario liquida un sueldo:



3.3 Diseño de DTOs y Mappers

Para evitar exponer directamente las Entities, el backend usa DTOs:

DTOs de Entrada:

- LiquidacionSueldoDTO
- EmpleadoDTO
- ConvenioDTO
- CategoriaDTO

DTOs de Salida

- PagoSueldoDetalleDTO
- EmpleadoDetalleDTO
- ConvenioDetalleDTO
- CategoriaDetalleDTO

📌 Los mappers traducen o convierten entre Entity y DTO para aislar capas.

3.4 Diseño de Seguridad

La versión actual utiliza:



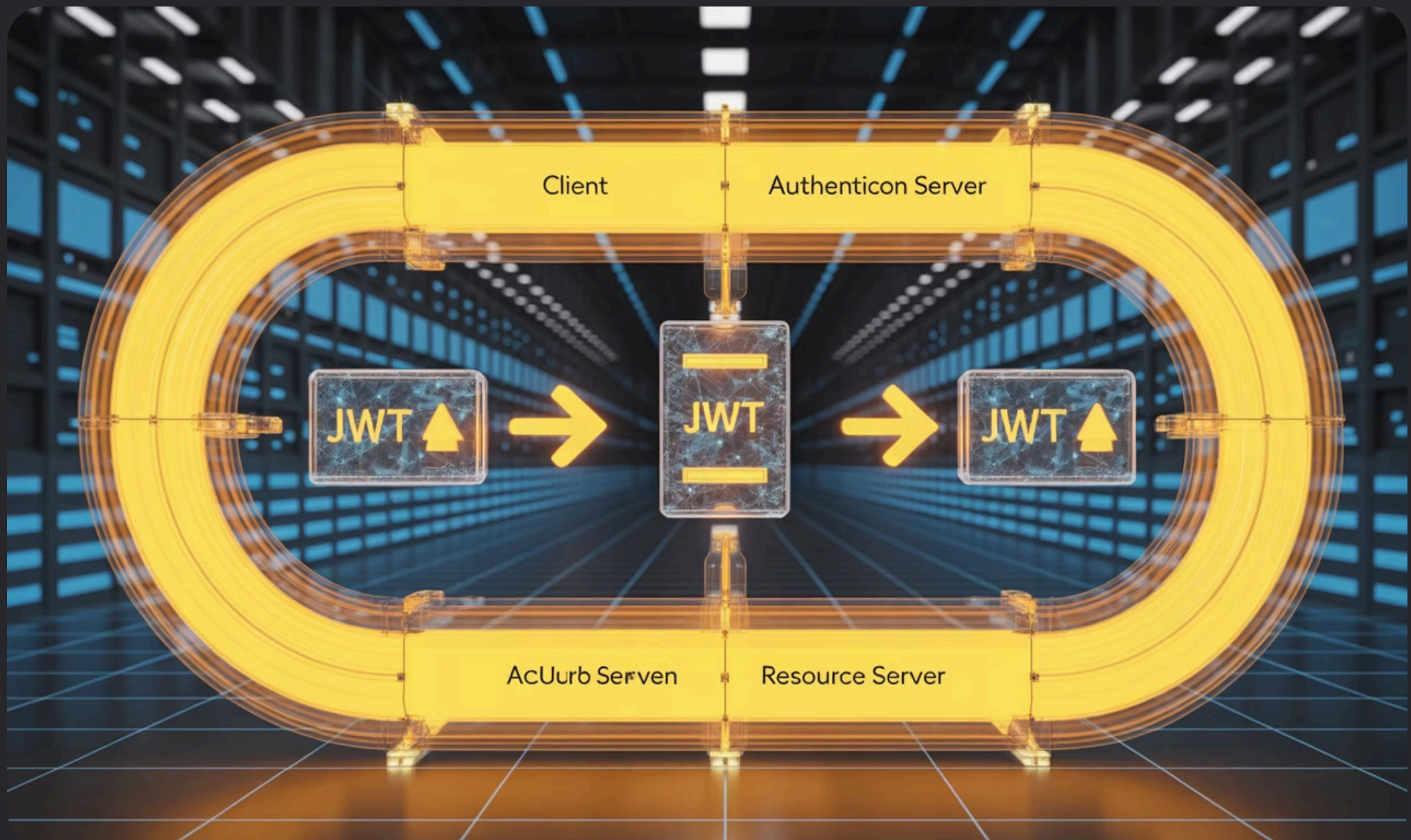
Autenticación JWT
activa



JwtUtil



CustomUserDetailsSer
vice



4 - Diseño de Base de Datos

El modelo de datos está basado en relaciones entre empleados, conceptos y liquidaciones.

4.1 Entidades Principales y Relaciones:

4.1 Empleado:

Datos personales y laborales.

Relaciones:

- Muchas a Una con Convenio
- Muchas a Una con Categoria
- Muchas a una con Zona
- Una a Muchas con PagoSueldo

4.2 Convenio:

Describe marco laboral (ej. Luz y Fuerza).

Tiene:

- Lista de Categoria
- Lista de bonificaciones y descuentos propios

4.3 Categoria

Asignada dentro de un convenio.

Define:

- Sueldo básico
- Antigüedad
- Valor hora

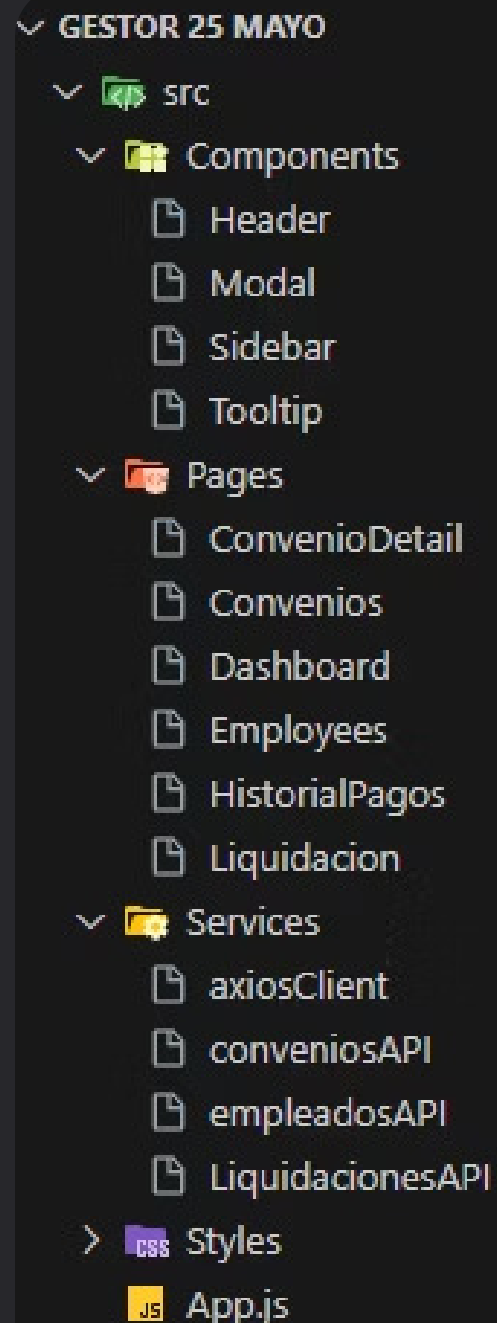
Entidades de Zona y Pagos

Zona	PagoSueldo	PagoConcepto
Define un porcentaje adicional según ubicación.	Registro de cada liquidación. Guarda: • Fecha • Periodo • Bruto • Descuentos • Neto	Cada concepto aplicado en la liquidación: • Tipo (bonificación/descuento) • Porcentaje o monto • Importe final Relación: Muchas a una con PagoSueldo



5 - Diseño del Frontend (Capa Cliente)

5.1 Estructura de Carpetas



```

GESTOR 25 MAYO
├── src
│   ├── Components
│   │   ├── Header
│   │   ├── Modal
│   │   ├── Sidebar
│   │   └── Tooltip
│   ├── Pages
│   │   ├── ConvenioDetail
│   │   ├── Convenios
│   │   ├── Dashboard
│   │   ├── Employees
│   │   ├── HistorialPagos
│   │   └── Liquidacion
│   ├── Services
│   │   ├── axiosClient
│   │   ├── conveniosAPI
│   │   ├── empleadosAPI
│   │   └── LiquidacionesAPI
│   ├── Styles
│   └── App.js

```

The image shows a file explorer window with a dark theme. The root folder is 'GESTOR 25 MAYO'. Inside it is a folder 'src'. Under 'src', there are four sub-folders: 'Components', 'Pages', 'Services', and 'Styles'. The 'Components' folder contains four files: 'Header', 'Modal', 'Sidebar', and 'Tooltip'. The 'Pages' folder contains six files: 'ConvenioDetail', 'Convenios', 'Dashboard', 'Employees', 'HistorialPagos', and 'Liquidacion'. The 'Services' folder contains four files: 'axiosClient', 'conveniosAPI', 'empleadosAPI', and 'LiquidacionesAPI'. The 'Styles' folder is currently collapsed. At the bottom of the 'src' folder is a file named 'App.js'.

Esta estructura permite modularidad y reuso.

5.2 Router y Navegación

Definido en App.js mediante react-router-dom:

- / → Dashboard
- /empleados → listado
- /empleados/ :id → ver detalles
- /liquidacion → proceso de liquidación
- /convenios
- /convenios/ :id
- /pagos y /pagos/ :id

5.3 Diseño de Componentes

La UI usa:

- Sidebar (menú lateral)
- Header (barra superior)
- Modales reutilizables
- Formularios con validaciones

Cada componente está desacoplado del estado global y se nutre de APIs desde services/.

5.4 Servicios de Comunicación (Axios)

- Timeout 10 segundos
- Manejo centralizado de errores

Servicios dedicados:

- empleadosAPI.js
- liquidacionesAPI.js
- conveniosAPI.js
- categoriasAPI.js

📄 Esto simplifica el mantenimiento y estandariza las llamadas.



5.5 Diseño de UX



Interfaz clara con
Sidebar fijo



Formularios
amplios y fáciles de
completar



Colores suaves y
componentes bien
espaciados



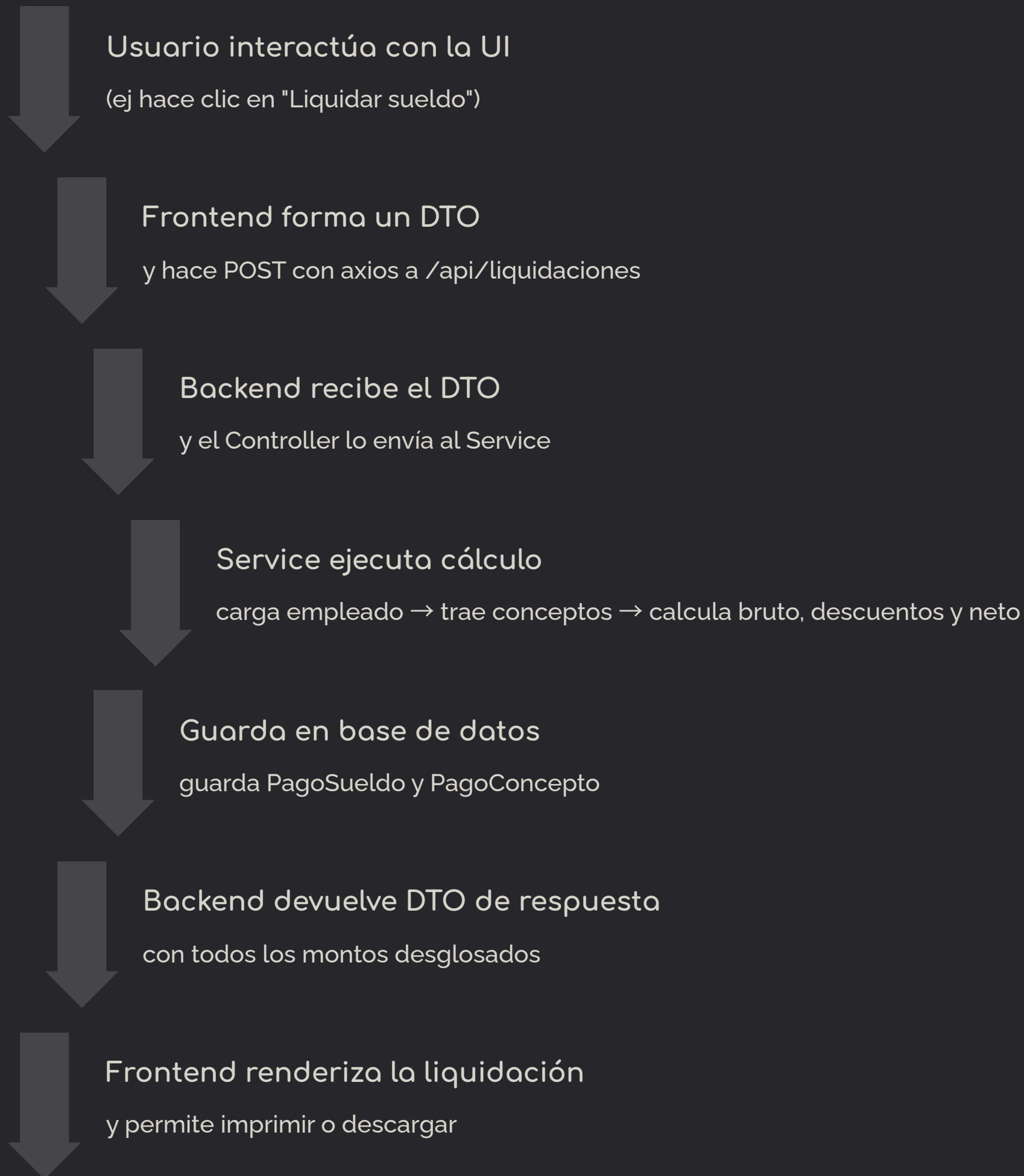
Animaciones sutiles
con Framer Motion



Modales intuitivos
para edición



6 - Flujo de Datos Completo



7 - Diseño de Errores y Validaciones

Backend

Validación con Jakarta Validation

- Excepciones controladas
- Respuestas JSON con mensajes claros

Frontend

Formularios con validación previa

- Toasts de error o éxito

Control de estados de carga ("loading")



Conclusión

Este diseño define cómo funciona internamente el Gestor de Sueldos:



Arquitectura clara y escalable



Backend robusto basado en Spring



Frontend modular y mantenible con React



Modelo de datos completo y bien normalizado



Flujo lógico consistente para la liquidación

Con este documento, el equipo cuenta con una guía técnica sólida de la estructura y decisiones de diseño que sostienen el sistema.

